Sample internal code (training data) is as follows:

```
# utils/validator.py
def validate_email(email: str) -> bool:
    """
    Validates an email address using internal business rules.
    """
    return "@" in email and email.endswith(".com")
def validate_user(user: dict) -> bool:
    """
    Ensures user object contains mandatory fields.
    """
    required_fields = ["id", "name", "email"]
    return all(field in user for field in required_fields)
```

Task: Explain what *validate_user* does.

Answer: It checks whether the user dictionary contains id, name, and email keys.

To prepare a fine-tuning dataset (simplified), type:

```
[
  {
    "prompt": "Explain the function validate_user.",
    "completion": "It checks whether the user dictionary
contains the keys id, name, and email."
  },
  {
    "prompt": "Write a function to validate phone number
using internal style.",
    "completion": "def validate_phone(phone: str) -> bool:\n
return phone.isdigit() and len(phone) == 10"
  }

]
```

This structure teaches the model:

- How internal functions are written
- Naming conventions
- Business validation logic

The training pipeline (conceptual code) is as follows:

```
from transformers import AutoModelForCausalLM, AutoTokenizer,
Trainer, TrainingArguments

model = AutoModelForCausalLM.from_pretrained("base-model")
tokenizer = AutoTokenizer.from_pretrained("base-model")

dataset = load_dataset("json", data_files="internal_data.
json")

training_args = TrainingArguments(
    output_dir="./fine_tuned_model",
    per_device_train_batch_size=2,
    num_train_epochs=3
)

trainer = Trainer(
    model=model,
    args=training_args,
    train_dataset=dataset["train"]
)

trainer.train()
```

This pipeline fine-tunes the model so that it understands:

- Internal utility functions
- Expected response style
- Domain logic

Evaluation tasks may include generating validation functions, explaining internal APIs, and suggesting refactoring. Engineers review model outputs for correctness and compliance with standards. Automated tests can measure consistency with internal logic.

The fine-tuned model can be integrated into IDEs and internal chat systems. Developers can ask: "Generate a validator for employee ID using company format." The model responds using learned conventions, ensuring consistent code generation.

All training and inference must occur in a secured internal environment. Proprietary algorithms must never leave organisational boundaries. Access controls and audit logs prevent misuse and data leakage.

Fine-tuned models improve productivity, reduce repetitive work, and help new employees understand internal systems. They also act as living documentation by encoding organisational knowledge.

But there are some risks, limitations, and ethical concerns. Models can inherit bad practices if trained on poorly written code. Over-automation may reduce human oversight. Governance policies must define acceptable usage and review requirements.

Fine-tuning a model on an internal codebase transforms AI into a domain-specific assistant. By preparing clean data, applying efficient training strategies, and integrating these securely into workflows, organisations can move from generic AI usage to highly specialised internal intelligence. This journey from zero to hero empowers developers rather than replacing them. END

By: Rajnesh Devi

The author is an assistant professor at the Yamuna Group of Institutions.